Sales app: what must change to match the web app

For the team maintaining `sales-mobile` (Flutter, package `stove_transaction`)

From ACSL, via the `sales-web` repository

Date 3 September 2026

Status requested work. Each item below carries the file and line in the mobile code, what the web app does today, and what the app should do to match.

> **Update, 3 September 2026, evening.** The web review is complete: every batch is merged and confirmed on production. Everything this brief called inherited has shipped, the contract below held, and the aggregate the brief promised for M8 exists: `get-sales-advanced` with `withSummary: true` returns a `summary` object beside the rows, computed in SQL and scoped to the caller. The items themselves are unchanged; nothing on this list has been closed from the web side.

At a glance

One line per item. The full finding, the file and line evidence, and what to do for each are in the item tables that follow.

Item	Group	What is wrong	Ask
M1	**Broken today**	Stove Management is broken for every role: `get-stove-ids` is called with an empty body and the server answers 400	Send `format: format2` or an organisation id and page with an offset loop; reuse the offline sync code
M2	**Broken today**	The Agreement PDF is built from the list row: `get-sale` is called as POST, the GET-only function fails, and the failure is swallowed	Call it as GET with `queryParams: {id}`; surface a failure
M3	**Broken today**	Every sale shows a green "Completed" badge, unpaid instalment and underpaid outright sales included	Derive the badge from `payment_status` and `total_paid` against `amount`
M4	**Broken today**	`fully_paid`, the value the server writes, has no label and prints raw	Add the case
M5	**Broken today**	Financial Reports prints `sales.status` raw; today every live sale reads "Incomplete"	Group by `payment_status`, or label the field for what it is
M6	**Broken today**	A part payment on an outright sale is accepted with no payment model, and the server records it as paid in full	Require a payment model unless amount received equals the amount; refuse on the field
M7	**Broken today**	Balance is shown only for instalment sales; an underpaid outright sale never shows what is owed	Show the balance whenever `total_paid` is below `amount`
M8	**Truncated on the phone**	Financial Reports totals and the dashboard drill-down sheets are summed on the phone from one 500-row fetch	Take totals from `get-sales-advanced` with `withSummary: true`; never sum rows to get a total
M9	**Truncated on the phone**	The sales list fetches 100 rows once (500 for a custom range) and pages on the phone; "a to b of N" reports the fetched count	Server offset paging; the total from `pagination.total`
M10	**Truncated on the phone**	Transfer history capped at 50, agents at 25, users at 25, a partner's stove sheet at 500; `has_more` is ignored	Page with `pagination.total` or `has_more`; copy the partner list
M11	**Correctness hygiene**	No timezone is pinned; every `DateFormat` and the "Today" filter use the handset's zone	One date helper pinned to `Africa/Lagos`, used everywhere
M12	**Correctness hygiene**	Profile deserialisation uses bare `DateTime.parse`; a malformed value kills the profile	`DateTime.tryParse` with a fallback
M13	**Correctness hygiene**	A failed photo upload during offline sync is not surfaced; the sale syncs without its photo	Report it, or hold the sale in the queue until the upload succeeds
M14	**Correctness hygiene**	Queued unsynced sales can vanish from the list on a decode error behind a bare catch	Log and show the failure; no bare catch around unsynced work
M15	**Correctness hygiene**	A failed profile refresh is invisible	A failure branch with a message
M16	**Platform**	The app version is a hardcoded constant kept in step with `pubspec.yaml` by hand	Read it from `package_info_plus`
M17	**Platform**	The update check bypasses the app's edge-function client and uses a raw HTTP GET	Route through `callEdgeFunction`
M18	**Platform**	User management is granted to agent managers and partners on the phone; the web grants it to super admins only	Match the web; change the two files together from now on
M19	**Platform**	Stale references in the permissions file and in the app's own rules file	Update the pointers
M20	**Platform**	The last direct table read is dead code with no caller	Delete it

Why this brief exists

The web app and the mobile app share one Supabase project and one set of edge functions, and nothing else. A review of the web app on 2 September 2026 found that several of its numbers were computed from truncated data, that some of its words did not match the data underneath, and that a few of its saves failed in silence. The web app is being fixed in ordered batches. This brief is the part of that review that only the mobile app can act on.

Two kinds of finding are separated on purpose:

- **Inherited.** Anything computed inside a shared edge function or in SQL reaches the mobile app the moment it deploys. The six dashboard cards the app shows (stoves received, sold, available; expected receivable, amount received, outstanding) come from the same three dashboard functions the web calls (`lib/features/dashboard/dashboard_service.dart:16-23`). When those functions move their sums into SQL, the app's cards become right with no app change. Nothing is asked of you for these.
- **Not inherited.** Anything the app renders or computes on the phone. The web cannot reach those. They are the twenty items below.

Facts you can rely on

- Production holds 2,145 live sales and 22,032 stoves as of the evening of 3 September 2026. Both grow daily.
- The app never reads tables directly; every read goes through an edge function and each function caps a single response: `get-sales-advanced` at 500 rows, `get-stove-ids` at 500, `get-transfer-history` at 200, `manage-organizations` at 100, `super-admin-agents` at 100, `manage-users` at 100. A screen that fetches once and sums or pages on the phone is therefore wrong or short as soon as the real count passes the cap, which for sales it already has.
- The database's own row limit (`max_rows`, 1,000) does not apply to the app, because the app does not query tables directly. Keep it that way.

The contract the web side commits to

So you can build against something stable:

1. Response shapes of the shared edge functions do not change.
2. Every list function keeps returning `pagination.total` or `has_more`, so the app can page rather than fetch once.
3. Done, 3 September: the dashboard functions compute every figure in SQL; the app's six cards are right with no app change.
4. New SQL report functions on the web side are service-role only and not reachable from the app. For sales totals the action already exists: send `withSummary: true` to `get-sales-advanced` with the same filters the list uses, and read `summary`: `total`, `receivable`, `collected`, `outstanding`, `fully_paid`, `partially_paid`, `unpaid`, and due (`overdue`, `dueToday`, `due7`, `due14`, `due30`). The rows still page with `page` and `limit` (500 at most) and `pagination.total` is exact. For any other screen that needs a server-side total, ask and an action will be exposed rather than the app summing rows.
5. `create-sale`, `update-sale` and `get-sale` are not being changed by the web review.

The items

Ordered by what matters most: broken or wrong today, then truncated, then correctness hygiene, then platform. The evidence column is the mobile repo unless a path starts with `supabase/`, which is the web repo.

Broken or wrong today

Item and finding	Evidence	What to do
M1 Stove Management is broken for every role. `get-stove-ids` is called with an empty body and the server answers 400 "Missing organization_id". The screen falls back to cache or throws	`lib/features/stoves/stoves_provider.dart:31-34` `supabase/functions/get-stove-ids/index.ts:40-50`	Send `format: format2` (or an organisation id) and page with an offset loop. The offline sync already does this correctly at `lib/core/services/data_refresh_service.dart:212-260`; reuse it
M2 The Agreement PDF is built from truncated list data. `get-sale` is called as POST with a JSON body; the function is GET and reads `?id=`. The call fails every time and the failure is swallowed, so the PDF silently uses whatever the list row held	`lib/features/sales/list/sales_list_screen.dart:1344-1352` `supabase/functions/get-sale/index.ts:4, 20`	Call it as GET with `queryParams: {id}`, exactly as `lib/features/sales/create/create_sale_service.dart:127` already does. Surface a failure instead of returning the list row
M3 Every sale shows a green "Completed" badge, including unpaid instalment sales and underpaid outright sales. The badge is derived from `is_installment` and `payment_status = partially_paid` only; everything else is "completed"	`lib/features/sales/list/sales_list_screen.dart:35-40`, used at `:698`, `:1105`, `:1183`	Derive the badge from `payment_status` and from `total_paid` against `amount`. The web's rule: `total_paid` is what was collected, for outright sales too, and is never substituted with `amount`. A server change cannot fix this badge
M4 `payment_status = fully_paid`, the value the server writes, has no label and prints raw as "fully_paid"	`lib/features/sales/list/sales_list_screen.dart:45-52`, `:1222` `supabase/functions/create-sale/index.ts:445, 579`	Add the case

Item and finding	Evidence	What to do
M5 Financial Reports prints `sales.status` raw as "Incomplete", "Pending", "Assigned". Those words are the sales form's completeness state, not payment, and today every live sale reads "Incomplete"	`lib/features/sales/financial_reports_screen.dart:89, 114`	Group by `payment_status`, or label the field for what it is. Do not present `sales.status` as a payment or verification state

M6
A part payment on an outright sale is accepted with no payment model, and the server then records it as paid in full (the server's outright path sets `total_paid` to the full amount)
	`lib/features/sales/create/create_sale_screen.dart:1601-1607`
	`lib/features/sales/create/create_sale_controller.dart:1672-1678`
	`supabase/functions/create-sale/index.ts:477-483`
		Apply the rule the web's bench applies at the form: a receipt with no payment model can only be saved when amount received equals the amount; otherwise a model is required. Refuse on the field, before submit

M7 Balance is shown only for instalment sales, so an underpaid outright sale never shows what is owed	`lib/features/sales/list/sales_list_screen.dart:1225-1226`	Show the balance whenever `total_paid` is below `amount`, whatever the type

Truncated on the phone

Item and finding	Evidence	What to do

M8
Financial Reports totals (count, value, collected, outstanding, by-status bars) and the dashboard drill-down sheets ("N sales, total", "N stoves") are summed on the phone from a single 500-row fetch. They are wrong today
	`lib/features/sales/sales_provider.dart:27`
	`lib/features/sales/financial_reports_screen.dart:82-93`
	`lib/features/dashboard/dashboard_screen.dart:597, :697, :750, :324, :462`
		Take totals from the server: `get-sales-advanced` with `withSummary: true` returns them under `summary` (see the contract), scoped and filtered exactly as the rows. Page rows only for lists, never to sum them

M9 The sales list fetches 100 rows once (500 for a custom date range) and pages them on the phone in 25s. Nothing past the first fetch is reachable, and "a to b of N" reports the fetched count, not the real one	`lib/features/sales/list/sales_list_controller.dart:24-36, :232-233` `lib/features/sales/list/sales_list_screen.dart:760`	Server offset paging with the page size the user picks, and the total from `pagination.total`

M10
Transfer history capped at 50, agents at 25, users at 25, a partner's stove sheet at 500; `has_more` is ignored and there is no paging control
	`lib/features/stoves/transfer_history_provider.dart:34`
	`lib/features/agents/agents_provider.dart:31-35`
	`lib/features/settings/users_provider.dart:29-33`
	`lib/features/partners/partners_screen.dart:525`
		Page with `pagination.total` or `has_more`. The partner list already does this correctly at `lib/features/partners/partner_access_service.dart:48-79`; copy it

Correctness hygiene

Item and finding	Evidence	What to do

M11
No timezone is pinned anywhere. Every `DateFormat` uses the handset's zone, and the "Today" filter uses the handset's local date, so a sale made late in the evening can land on the wrong day
	`lib/features/sales/list/sales_list_screen.dart:22, 866`
	`lib/features/dashboard/dashboard_controller.dart:33`
	`lib/features/sales/list/agreement_pdf_generator.dart:29`
	`lib/features/sales/list/sales_list_controller.dart:202-214`
		One date helper pinned to `Africa/Lagos`, used everywhere, including filter boundaries

M12 Profile deserialisation uses bare `DateTime.parse`; a malformed value throws and kills the profile	`lib/core/model/profile_model.dart:42, 108`	`DateTime.tryParse` with a fallback
M13 During offline sync, a failed photo upload is not surfaced; the sale syncs without its photo and nobody is told	`lib/core/services/sync_service.dart:91`	Report it to the user, or hold the sale in the queue until the upload succeeds
M14 Queued unsynced sales can vanish from the list on a decode error, because the load is wrapped in a bare catch	`lib/features/sales/list/sales_list_controller.dart:173`	Log and show the failure; never a bare catch around unsynced work
M15 A failed profile refresh is invisible	`lib/features/profile/profile_controller.dart:75-98`	A failure branch with a message

Platform

Item and finding	Evidence	What to do
M16 The app version is a hardcoded constant kept in step with `pubspec.yaml` by hand	`lib/core/services/app_release_service.dart:9` `pubspec.yaml:6`	Read it from `package_info_plus`
M17 The update check bypasses the app's own edge-function client and calls the function with a raw HTTP GET	`lib/core/services/app_release_service.dart:70-71`	Route through `callEdgeFunction` like every other call
M18 User management is granted to agent managers and partners on the phone; the web grants it to super admins only	`lib/core/auth/permissions.dart:123, 177` web `src/lib/permissions.ts:106`	Match the web. The two files are one rule and must be changed together from now on

M19
Stale references: the permissions file points at a repository path that no longer exists, and the app's own rules file still names a direct-table exception that has since been removed
	`lib/core/auth/permissions.dart:6, 100`
	`CLAUDE.md` hard rules
	`lib/features/partners/partner_access_service.dart:11-12`
		Update the pointers

M20 The last direct table read is dead code with no caller	`lib/core/services/supabase_service.dart:285-307`	Delete it, so the "edge functions only" rule can be proven by search

Two things the app does better than the web, and should keep

- It requires the buyer's signature at the point of sale (`lib/features/sales/create/create_sale_controller.dart:1617`). Right for a field sale, where the buyer is present.
- It mirrors the partner payment-model rule exactly: an explicit list restricts, no list means every active model (`lib/features/sales/create/create_sale_controller.dart:282-287`), the same rule the server enforces.

What we would like back

- One pull request per group above (broken, truncated, hygiene, platform), or one per item where the item is large, against `sales-mobile`, each naming the M numbers it closes.
- For M1 to M10, a before and after on a test build: the screen, the number or the badge, and the request the app made. For M11, a sale saved after 23:00 West Africa Time appearing under the right day.
- Any item you believe is wrong or already fixed, said so with the evidence, rather than skipped.

Questions go to ACSL through the usual channel and are answered in writing so the answer travels with the work.